I. Decidability & Recognizability

- **1. The Enumerator Trap**: **Q:** Does an enumerator exist that prints machines for every recognizable language over **{0,1}**, but those machines *never* halt on rejected words? **A:** Yes. We don't try to solve the Halting Problem to find them. Instead, we enumerate all TMs, and blindly replace any transition to q_{reject} (or undefined crashes) with an infinite loop. **Key Insight:** You don't have to analyze machine behavior to build an enumerator. You can just algorithmically modify source code to force the behavior you want while preserving the language.
- **2. The Mixed Language Recognizer**: **Q:** If $L \in \mathrm{RE}$ and $\overline{L} \notin \mathrm{RE}$, is $L' = \{0w \mid w \in L\} \cup \{1w \mid w \notin L\}$ recognizable? **A:** No. If L' is recognizable (via machine M_1), we can build a recognizer for $\overline{L}$ by simply feeding $1w$ into M_1. Since $\overline{L}$ is strictly not recognizable, this is a contradiction. **Key Insight:** Prove undecidability implicitly by showing that a decider/recognizer for a new language could be used as a trivial subroutine to solve a known impossible language.

II. Mapping Reductions

- **3. The Halting-to-Acceptance Blueprint**: **Q:** Show a mapping reduction $HALT_{\mathsf{TM}} \leq_m A_{\mathsf{TM}}$. **A:** The function $f(\langle M, w\rangle)$ outputs M'. M' runs M on w. If M accepts, M' accepts. If M explicitly rejects, M' accepts. **Key Insight:** In a mapping reduction, f just writes "dumb" source code. Never assume a decider exists inside the generated dummy machine. Halting simply means catching both the accept and reject states.
- **4. The Meta Halting Set**: **Q:** Prove $DEC\text{-}HALT_{\mathsf{TM}}$ is not Turing-Recognizable. **A:** Map from $\overline{A_{\mathsf{TM}}}$. If invalid input, output a machine that accepts everything (Σ^*, decidable). O/W, output M' that runs M on w. If M rejects, loop forever (Halting set = $\emptyset$). If M accepts, run the A_{TM} recognizer on input (Halting set = A_{TM}). **Key Insight:** To prove not in RE, map from $\overline{A_{\mathsf{TM}}}$. Tie the Accept/Reject behavior of M to Decidable ($\emptyset$, Σ^*) or Undecidable (A_{TM}) sets.
- **5. The Palindrome Property**: **Q:** Prove T (machines that accept w^R whenever they accept w) is not RE. **A:** Map from $\overline{A_{\mathsf{TM}}}$. Output M' that runs M on w. If it loops, $L(M') = \emptyset$ (satisfies T). If it accepts, M' acts as strict regular expression for $(01)^+$ (violates T because **10** is not accepted). **Key Insight:** Choose anchor languages wisely. Pick the simplest possible language that definitively proves or breaks the property you are testing.

III. Time Complexity & P vs. NP

- **6.** 🚩 **The Verifier Masterclass**: **Q:** Prove $PRIME\text{-}FACTOR \in \mathrm{NP} \cap \mathrm{coNP}$. **A:** The certificate is the prime factorization. NP verifier checks primality, verifies the product equals n, and any factor is $\leq m$. coNP verifier does the same math validation, but checks if *all* factors are $> m$. **Key Insight:** For coNP verifiers, outline that you cannot just "flip the output" of the NP verifier. You must fully validate that the certificate is mathematically honest before testing the NO condition.
- **7. Math Triviality vs. NP-Complete**: **Q:** $HALF\text{-}3SAT$ (satisfy half of clauses) vs $FIRST\text{-}HALF\text{-}3SAT$ (satisfy first half exactly). **A:** $FIRST\text{-}HALF$ is NPC via padding (add dummy clauses with new vars to second half). $HALF\text{-}3SAT$ is in P because a random guess satisfies **7/8** of 3-CNF clauses, so every valid formula always has a "YES" answer. The algorithm just checks syntax. **Key Insight:** Sometimes problems are in P not from clever search, but because math guarantees the answer is always YES.
- **8. Graph Padding Reduction**: **Q:** Reduce $CLIQUE$ to $HALF\text{-}CLIQUE$. **A:** If $k > n/2$, pad with isolated nodes. If $k < n/2$, pad with a fully connected clique of $n - 2k$ nodes, connected to every node in the original. **Key Insight:** Physically alter structure (padding with disconnected or fully connected nodes) to force the new parameter to equal the old one.

IV. Automata Mechanics & Closures

- **9. The Counter DFA**: **Q:** Build a counter DFA for $a^i b^j a^k c$ where $i + k > j$. **A:** Inverse counting: increment for b's and decrement for a's. Accept only if counter is negative (state **n**) when final **c** is read. **Key Insight:** When constrained by limited hardware, re-arrange the algebra of the inequality to fit the hardware.
- **10. Formal DFA Concatenation**: **Q:** Formally define DFA for $L(A) \cdot \{cc\} \cdot L(B)$. **A:** Define a new 5-tuple. Add bridge states to track the two **c**'s. Explicitly define q_{sink} to catch any premature **c**'s or invalid symbols for totally defined transitions. **Key Insight:** Formal definitions require absolute precision. If you expand the alphabet, you must define exactly where the new letters go for every single existing state.
- **11.** 🚩 **The Inequality Trap Pumping Lemma**: **Q:** Prove $a^j b^m$ where $j \neq 2m$ is not regular. **A:** Choose $s = a^{p!} b^{p!}$. Because $p!$ is a universal numerator, no matter y's length k, you can perfectly divide it to pump a's to $2(p!)$. **Key Insight:** When fighting inequality ($\neq$), pump *into* the equality. Use Factorials ($p!$) to guarantee perfect integer division when calculating the multiplier i. When the demon picks $|y| = k$, pump exactly $i = (p!/k) + 1$ times
- **12. State-Space Closure**: **Q:** Prove $\tilde{L} = \{uv \mid u^R v \in L\}$ is regular. **A:** Use original DFA states as halfway meeting points (p). Prove U_p regular (reversal closure), V_p regular, and concatenation L_p regular. Final language is finite union of all L_p. **Key Insight:** For infinite paths or word splits in a DFA, anchor proofs to the **finite states**. Finite unions of regular sub-languages preserve regularity.

V. Context-Free Languages & PDAs

- **13. CFG Reverse Engineering**: **Q:** Analyze $B \to aBb \mid C$ and $C \to aC \mid \varepsilon$, prove ambiguity. **A:** Language generated is $a^m b^n$ where $m \geq n$, because C rule injects a's into the *middle*, merging with left-hand a's. Ambiguity proven by drawing two distinct leftmost derivations for **aa**. **Key Insight:** CFG rules build from "outside in." Trace where variables physically drop terminals relative to boundaries.
- **14. CFL Pumping Lemma**: **Q:** Prove $a^i b^{2i-j} c^j$ ($2 \leq j < i$) is not CFL. **A:** Choose hair-trigger string $s = a^{p+3} b^{p+4} c^{p+2}$. Define cases strictly by where the window vxy falls (entirely in one section, or crossing a boundary). **Key Insight:** Define CFL pumping cases by the span of the vxy window, not contents of v and y. Eliminates loophole of v or y being empty. Ensure lengths are 1 unit apart so a single pump breaks inequality.

VI. The Foundations

- **15. Proof by Diagonalization**: **Q:** Prove a language like A_{TM} or $HALT_{\mathsf{TM}}$ is undecidable from scratch. **A:** Assume perfect decider H exists. Build spiteful dummy D taking $\langle M\rangle$. It feeds $\langle M, \langle M\rangle\rangle$ to H. D then conditionally does the *opposite* of H's prediction. Feed D its own source ($\langle D\rangle$); D accepts iff D rejects, a paradox. **Key Insight:** Diagonalization is weaponized spite. You assume a perfect observer exists, ask for its prediction, and intentionally defy it. Creates a paradox that proves observer cannot exist.

VII. Execution Mechanics & Exam Hacks

16. The "Dovetailing" (Parallel Simulation) Cheat Code

- **Problem & Solution:** You need to prove a language is Recognizable (in RE), but testing a machine might result in an infinite loop. 🚩 You cannot run M_1 to completion, then M_2. You must "Dovetail."** Run M_1 for 1 step. Run M_1, M_2 for 2 steps. Run M_1, M_2, M_3 for 3 steps.
- **When to use:** Building Enumerators from Recognizers, simulating multiple TMs, or proving union/intersection closures for RE.

17. Rice's Theorem: Prove L is Undecidable instantly

- **The Rule:** Any non-trivial property of the *language* recognized by a TM is undecidable. If P is non-trivial, then $LP = \{\langle M\rangle | L(M) \in P\}$ is Undecidable. 🚩 **The Trap:** The property MUST be about $L(M)$, not the physical machine M. $L(M1) = L(M2) \Leftrightarrow (M1 \in L \wedge M2 \in L) \vee (M1 \notin L \wedge M2 \notin L)$. -✅ *Undecidable (Rice applies):* "Does M accept '101'?" / "Is $L(M)$ regular?". 🚩 *Decidable (Rice FAILS):* "Does M have 5 states?" / "Does M halt in < 100 steps?"(checkable in finite time). **Extended Rice (Not in RE):** If the property P is true for the empty set ($L(M) = \emptyset$), then any language that *requires* accepting strings to satisfy P is **Not in RE** (because you can reduce from $\overline{A_{TM}}$ where the looping case results in $\emptyset$).

18. PDA & Stack Architecture Patterns

- **Equality ($i = j$):** Push A's, pop for B's.
- **Multiples ($i = 2j$):** Push TWO A's for every input A. Pop ONE for every B. (Or vice versa).
- **Inequality ($i \neq j$):** Non-determinism is required! Branch the machine at the start: Branch 1 guesses $i > j$, Branch 2 guesses $i < j$.
- **Adjacent Monotonicity ($k_{n-1} < k_n$):** To compare the final two blocks, push C's for Block $n - 1$. Use an ϵ-transition (no input consumed) to move to the final block. Pop C's for Block n. If the stack hits \$ while input still remains, transition to the accept state to guarantee $k_n > k_{n-1}$.

19. NP-Completeness Reduction Arsenal (Which to choose?)

Don't guess which NP-Complete problem to reduce from. Match the structure:

- **Graph Structure / Connecting things:** Reduce from `CLIQUE`, `INDEPENDENT-SET`, or `VERTEX-COVER`.
  - *Hack:* $VC = \text{Total Nodes} \setminus IS$. If you find an Independent Set of size k, the remaining nodes form a Vertex Cover of size $V - k$.
  - *The Holy Trinity Hack:* `CLIQUE` in G of size $k \iff$ `INDEPENDENT-SET` in $\overline{G}$ (the complement graph) of size $k \iff$ `VERTEX-COVER` in $\overline{G}$ of size $|V| - k$. You can trivially swap between these three!
- **Routing / Traversing:** Reduce from `HAM-CYCLE` or `HAM-PATH`.
  - *The Cycle-to-Path Hack:* To reduce Cycle to Path, pick an arbitrary node v. Split it into two nodes, v_{start} and v_{end}.

Connect v_{start} to all of v's original outgoing edges, and v_{end} to all incoming edges. A Hamiltonian Path from v_{start} to v_{end} in the new graph strictly corresponds to a Hamiltonian Cycle in the old graph.

- **Sets / Grouping / Numbers:** Reduce from `SUBSET-SUM`, `PARTITION`, or `SET-COVER`.
  - *The Target Forcing Hack:* If converting `SUBSET-SUM` (target T) to `PARTITION` (equal halves), add one or two massive "dummy" items to the set. Size them perfectly so that halving the new grand total mathematically *forces* the original items to sum to exactly T on one side.
- **Logic / Rigid Constraints:** Reduce from `3SAT`. (Variables = choices, Clauses = rules).
  - *The 3SAT Gadget Hack:* Variables always map to a pair of mutually exclusive nodes (e.g., connected by an edge so you can only pick one: x or $\neg x$). Clauses map to a triangle/group of nodes where the problem's rules force you to select at least one "True" node to cover the clause.
- **The Padding Trick:** If reducing $k \to f(n)$ (e.g., `CLIQUE` to `HALF-CLIQUE`), physically pad the graph with disconnected nodes (to dilute the ratio) or a fully-connected subgraph (to artificially inflate the clique size).

VIII. The "Red Flag" Trap Checklist

If you see these phrases on the exam, alarms should go off in your head:

- **"TM that halts in at most $|<M>|$ steps"**: *Trap:* It looks like the Halting Problem, but it's bound by a finite number. You can just simulate it for exactly that many steps. It is **Decidable**.
- **"For all x..." vs "There exists an x..." in reductions**: *Trap:* If a property requires checking *all* strings (e.g., $L(M) = \Sigma^*$), it is usually not in RE (requires infinite verification). If it requires finding *one* string ($L(M) \neq \emptyset$), it is usually in RE (can dovetail to find it).

IX. Additional High-Yield Exam Problems

20. Decidability of DFA Graph Properties (`MMN6` - Q1)

- **Q:** Is $LONGEST_{DFA} = \{\langle A, w\rangle \mid w \text{ is the longest word in } L(A)\}$ decidable?
- **A:** Yes. A DFA has a longest word if and only if there are NO cycles on any path from the start state to an accept state. The decider checks for cycles (using DFS/BFS). If a cycle exists, it rejects. If no cycle exists, the DFA is a finite Directed Acyclic Graph (DAG), and it simply finds the longest path to an accept state and checks if it equals w. **Key Takeaway:** When checking decidability for DFAs, treat the DFA as a directed graph. Checking for infinite languages = checking for cycles.

21. NP-Complete Edge Gadgets (`SampleExam` - Q4a)

- **Q:** Reduce `VERTEX-COVER` to `DOMINATING-SET`.
- **A:** For every edge $e = (u, v)$ in the original graph, add a new "gadget" node x_e connected *only* to u and v. To dominate this isolated x_e, the dominating set is physically forced to pick either u, v, or x_e. Picking u or v is always strictly better as it covers other edges too. **Key Takeaway:** If you need to force an NP-Complete choice (like "pick at least one endpoint of every edge"), create a dead-end "gadget" node attached to those endpoints.

22. The $O(n)$ Multi-Tape Algorithm (`MMN5` - Q5)

- **Q:** Build a 2-tape TM that decides $D = \{ww\}$ in strictly $O(n)$ time.
- **A:** Tape 1 has two pointers: a "fast" pointer moving 2 steps per cycle, and a "slow" pointer moving 1 step. When the fast pointer hits the end, the slow pointer is exactly in the middle. Copy the second half to Tape 2, rewind Tape 1 to the start, and compare Tape 1 to Tape 2 simultaneously. **Key Takeaway:** To find the middle of a string in linear time without sweeping back and forth (which costs $O(n^2)$), use the "fast/slow pointer" trick on a multi-tape machine.

23. The Un-Recognizability of Loops (`moedA1` - Q3b)

- **Q:** Is $Loop(v) \setminus \{v\}$ recognizable? (Where $Loop(v)$ is all TMs that infinite-loop on a specific input v).
- **A:** No. $Loop(v)$ is mathematically equivalent to $\overline{HALT_{TM}}$ (just hardcoded for input v), which is known to be Not in RE. Subtracting a single finite string $\{v\}$ from an infinite non-RE set cannot magically make it recognizable. **Key Takeaway:** You cannot build a recognizer for machines that *don't* halt. Removing a finite number of edge cases from an un-recognizable set leaves it un-recognizable.

X. Closure for R/RE and Key Theorems

Op	R (Decidable)	RE
Union / Concat / *	✓ Yes	✓ Yes
Intersect	✓ Yes	✓ Yes
Complement	**✓ Yes**	**X NO**

Note: If RE were closed under complement, then $RE = R$ (This is False).

1. $R = RE \cap \text{co-}RE$ (If L and $\bar{L}$ are recognizable $\to$ decidable).
2. $A_{TM} \in RE \setminus R$ (Halting Problem).
3. $\overline{A_{TM}} \in \text{co-}RE \setminus RE$ (Not even recognizable!).

XI. CFG: Prefix/Suffix Logic

Language: $L = \{cw_1 \dots cw_k \mid k \geq 3, \exists i : \#_a(w_i) = 2\#_b(w_{i+2}) + 1\}$

Rule:

- $S \to PcMQ$ (Structure: Prefix, Triple, Suffix)
- $P, Q \to cWP \mid \varepsilon$ (Generates generic blocks cw)
- $M \to BaBaMAbA \mid BaBcWcA$ (Recursion: $2a \in w_i \leftrightarrow 1b \in w_{i+2}$)
- $W \to aW \mid bW \mid \varepsilon$ (Base: Adds last $+1a, w_{i+1}$)
- $A \to aA \mid \varepsilon, B \to bB \mid \varepsilon$ (Free chars: $b \in w_i, a \in w_{i+2}$)

M-Rule Breakdown:

- **Rec ($BaBaMAbA$):** Pairs $2 \times a$ in w_i with $1 \times b$ in w_{i+2} for each step.
- **Base ($BaBcWcA$):** Stops recursion, adds the required $+1$ 'a' to w_i, and generates w_{i+1}.
- **Vars (A, B):** Allow "uncounted" chars ($b \in w_i, a \in w_{i+2}$) to fulfill $w \in \{a, b\}^*$.

XII. CFG: Distant Dependency

Language: $L = \{dw_1 \dots w_n d \mid n \geq 4, \exists k \in [2, n-2] : |w_k| = \#_c(w_{k+2})\}$

Rules:

- $S \to dPdMQd$ (Structure: Ensure d delimiters)
- $P \to WdP \mid W$ (Prefix: ≥ 1 block $w_1 \dots w_{k-1}$)
- $Q \to dWQ \mid \varepsilon$ (Suffix: blocks after w_{k+2})
- $M \to aMVcV \mid bMVcV \mid cMVcV \mid dWdV$ (Match $|w_k|$ to $\#_c(w_{k+2})$)
- $W \to aW \mid bW \mid cW \mid \varepsilon$ (Any string in $\{a, b, c\}^*$)
- $V \to aV \mid bV \mid \varepsilon$ (Free a, b in w_{k+2}; no c's)

Logic: P ensures $k \geq 2$ by generating at least w_1. M recursively pairs any char in w_k with a 'c' in w_{k+2}. V allows non-'c' chars in w_{k+2}. Base of M creates middle block w_{k+1}.

XIII. CFG: Examples

- $L_1 = \{w \in \{a, b\}^* \mid \#_b(w) \bmod 3 = 1\}$
  $S \to aX \mid bS, \quad X \to aY \mid bX \mid \varepsilon, \quad Y \to aS \mid bY$
- $L_2 = \{a^n b^n \mid n \geq 0\}$
  $S \to aSb \mid \varepsilon$
- $L_4 = \{a^{3n} b^{n+1} \mid n \geq 0\}$
  $S \to aaaSb \mid b$
- $L_5 = \{a^m b^k \mid m > k\}$
  $S \to aSb \mid aS \mid a$
- $L_6 = \{a^m b^k \mid m \neq k\}$
  $S \to aSb \mid A \mid B, \quad A \to aA \mid a, \quad B \to bB \mid b$
- $L_7 = \{a^m b^k c^{2m+k} \mid m, k > 0\}$
  $S \to aScc \mid aXcc, \quad X \to bXc \mid bc$
- $L_8 = \{a^m b^{m-k} c^k \mid m > k\}$
  $S \to aSc \mid X, \quad X \to aXb \mid ab$
- $L_9 = \{a^m b^k \mid k \leq m \leq 3k\}$
  $S \to aSb \mid aaSb \mid aaaSb \mid \varepsilon$
- $L = \{a^m b^k \mid m, k \geq 0, m \leq k \leq 3m\}$
  $S \to aSb \mid aSbb \mid aSbbb \mid \varepsilon$
- $L_{10} = \{a^m b^k \mid k < m < 2k\}$
  $S \to aSb \mid aaSb \mid aaabb$
- $L_{11} = \{a^m b^m a^k b^k \mid m, k \geq 0\}$
  $S \to XX, \quad X \to aXb \mid \varepsilon$
- $L_{12} = \{a^m b^s c^k d^t \mid m + s = k + t\}$
  $S \to aSd \mid X \mid Y, \quad X \to aXc \mid Z, \quad Y \to bYd \mid Z, \quad Z \to bZc \mid \varepsilon$
- $L_{13} = \{a^m b^s c^k d^t \mid m + t = s + k\}$
  $S \to XY \mid MN, \quad M \to aXb \mid \varepsilon, \quad N \to bNd \mid Y, \quad Y \to cYd \mid \varepsilon,$
- $L_{14} = \{w \in \{a, b\}^* \mid w^R = w\}$
  $S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$
- $L_{15} = \{w \in \{a, b\}^* \mid w^R \neq w\}$
  $S \to aSa \mid bSb \mid aXb \mid bXa, \quad X \to aX \mid bX \mid \varepsilon$
- $L_{16} = \{w \in \{a, b\}^* \mid w^R = w, \#_a(w) \bmod 3 = 1\}$
  $S \to bSb \mid aXa \mid a, \quad X \to bXb \mid aYa, \quad Y \to bYb \mid aSa \mid b \mid \varepsilon$
- $L_{17} = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$
  $S \to aSb \mid bSa \mid SS \mid \varepsilon$
- $L_{18} = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w) + 1\}$
  $S \to XaX, \quad X \to aXbX \mid bXaX \mid \varepsilon$
- $L_{19} = \{w \in \{a, b\}^* \mid \#_a(w) = 2 \cdot \#_b(w)\}$
  $S \to aSaSbS \mid aSbSaS \mid bSaSaS \mid \varepsilon$
- $L = \overline{\{ww \mid w \in \{a, b\}^*\}}$
  $S \to S_1 \mid S_2, \quad S_1 \to TTS_1 \mid T, \quad S_2 \to XY \mid YX, \quad X \to TXT \mid a,$
- $L = \overline{\{a^n b^n c^n \mid n \geq 0\}}$
  $S \to S_1 \mid S_2 \mid S_3 \mid S_4 \mid S_5, \quad S_1 \to XbXaX, \quad X \to aX \mid bX \mid cX \mid \varepsilon,$
- $L_{20} = \{a^m b^n c^p \mid m + 2n \geq p\}$
  $S \to aSc \mid aS \mid A, \quad A \to bAcc \mid bAc \mid bA \mid \varepsilon$
- $L_{21} = \{a^m b^n \mid 3m \leq 5n \leq 4m\}$
  $S \to a^4 Ab^3 \mid a^3 Ab^2 \mid a^7 Ab^5 \mid a^6 Ab^4 \mid A, \quad A \to a^5 Ab^3 \mid a^5 Ab^4 \mid \varepsilon$